

cms_doubleelectron_ppee_QED0_XL

June 6, 2026

1 OTUS | CMS DoubleElectron with MG5 Z/γ^* Prior

This notebook trains OTUS using CMS Open Data DoubleElectron events as detector-level data x , and a standalone MadGraph5 pre-detector Drell–Yan $pp \rightarrow e^+ e^-$ prior as latent/theory-level data z .

Expected z HDF5 file layout:

```
FDL/zData : shape (N, 8)
columns   : [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]
```

Put the MG5 HDF5 file in the same folder as this notebook, or set `theory_prior_file` to its full path in the meta-parameter cell.

2 Load Required Libraries

```
[17]: %matplotlib inline
from pathlib import Path

import awkward as ak
import matplotlib.pyplot as plt
import numpy as np
import torch
import uproot

import sys
from pathlib import Path

import os

os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

# Change this to your actual OTUS root folder
OTUS_ROOT = Path(r"C:\Users\AhrixMarin\Desktop\otus")
UTILITY_DIR = OTUS_ROOT / "utilityFunctions"
```

```

assert (UTILITY_DIR / "func_utils.py").exists(), f"Cannot find func_utils.py in {UTILITY_DIR}"

sys.path.insert(0, str(UTILITY_DIR))

print("Using utilityFunctions from:", UTILITY_DIR)

## Determine if using GPU or CPU ##
import os
os.environ['CUDA_VISIBLE_DEVICES'] = '0' # Set to '-1' to disable GPU

from configs import device

print('Using device:', device)

```

Using utilityFunctions from: C:\Users\AhrixMarin\Desktop\otus\utilityFunctions
Using device: cuda

3 Meta Parameters

```

[18]: cms_root_file = Path('../cms_zpeak/data/Run2012B_DoubleElectron.root')

# MG5 prior produced on Linux and copied to this notebook folder.
# Change this filename if your HDF5 file has a different name.
theory_prior_filename = 'cms_dyee_mg5_8tev_fiducial_70_110.hdf5'
theory_prior_file = Path(theory_prior_filename)

# Used only for saved model/results filenames.
dataset_name = 'cms_doubleelectron_mg5_dyee_XL_lambda0.7'

## Set random seeds ##
seed = 0
torch.manual_seed(seed)
np.random.seed(seed)

## Set data type ##
from configs import float_type
print('Using data type:', float_type)
print('Notebook/kernel working directory:', Path.cwd())
print('Expected MG5 prior file:', theory_prior_file)
print('Local .hdf5 files:', sorted([p.name for p in Path('.').glob('*.hdf5')]))

```

Using data type: float32
Notebook/kernel working directory:
c:\Users\AhrixMarin\Desktop\otus\experiments\cms_doubleelectron
Expected MG5 prior file: cms_dyee_mg5_8tev_fiducial_70_110.hdf5
Local .hdf5 files: ['cms_dyee_mg5_8tev_dy1j_ptj5_fiducial_70_110.hdf5',

```
'cms_dyee_mg5_8tev_fiducial_70_110.hdf5']
```

4 Load CMS DoubleElectron ROOT File

```
[19]: assert cms_root_file.exists(), f'File not found: {cms_root_file.resolve()}'
```

```
cms_file = uproot.open(cms_root_file)
events = cms_file['Events']

print('File:', cms_root_file.resolve())
print('Number of events:', events.num_entries)

electron_branches = [k for k in events.keys() if 'Electron' in k]
electron_branches[:30]
```

File: C:\Users\AhrixMarin\Desktop\otus\experiments\cms_zpeak\data\Run2012B_DoubleElectron.root

Number of events: 21474287

```
[19]: ['nElectron',
      'Electron_pt',
      'Electron_eta',
      'Electron_phi',
      'Electron_mass',
      'Electron_charge',
      'Electron_pfRelIso03_all',
      'Electron_dxy',
      'Electron_dxyErr',
      'Electron_dz',
      'Electron_dzErr']
```

```
[20]: branches = [
      'nElectron',
      'Electron_pt',
      'Electron_eta',
      'Electron_phi',
      'Electron_mass',
      'Electron_charge',
      'Electron_pfRelIso03_all',
      'Electron_dxy',
      'Electron_dz',
  ]

arrays = events.arrays(branches, library='ak')

print('Loaded events:', len(arrays['nElectron']))
print('Example nElectron values:', arrays['nElectron'][:10])
```

Loaded events: 21474287

Example nElectron values: [0, 1, 2, 2, 1, 2, 0, 2, 1, 0]

5 Select Good Electron Pairs

```
[21]: # Basic kinematic cuts
electron_pt_min = 20.0
electron_abs_eta_max = 2.5

# Electron quality cuts
electron_iso_max = 0.15
electron_dxy_max = 0.05
electron_dz_max = 0.10

# Z-window cut
z_mass_min = 70.0
z_mass_max = 110.0

electrons = ak.zip({
    'pt': arrays['Electron_pt'],
    'eta': arrays['Electron_eta'],
    'phi': arrays['Electron_phi'],
    'mass': arrays['Electron_mass'],
    'charge': arrays['Electron_charge'],
    'pfRelIso03_all': arrays['Electron_pfRelIso03_all'],
    'dxy': arrays['Electron_dxy'],
    'dz': arrays['Electron_dz'],
})

abs_eta = np.abs(electrons.eta)
outside_ecal_gap = ~((abs_eta > 1.4442) & (abs_eta < 1.566))

selected_electrons = electrons[
    (electrons.pt > electron_pt_min)
    & (abs_eta < electron_abs_eta_max)
    & outside_ecal_gap
    & (electrons.pfRelIso03_all < electron_iso_max)
    & (np.abs(electrons.dxy) < electron_dxy_max)
    & (np.abs(electrons.dz) < electron_dz_max)
]

print('Total selected electrons:', ak.sum(ak.num(selected_electrons)))
print('Events with >=2 selected electrons:', ak.sum(ak.num(selected_electrons)
    ↪ >= 2))

pairs = ak.combinations(selected_electrons, 2, fields=['e1', 'e2'])
os_pairs = pairs[(pairs.e1.charge * pairs.e2.charge) < 0]
```

```

e_minus = ak.where(os_pairs.e1.charge < 0, os_pairs.e1, os_pairs.e2)
e_plus = ak.where(os_pairs.e1.charge > 0, os_pairs.e1, os_pairs.e2)

def pair_mass(first, second):
    px = first.pt * np.cos(first.phi) + second.pt * np.cos(second.phi)
    py = first.pt * np.sin(first.phi) + second.pt * np.sin(second.phi)
    pz = first.pt * np.sinh(first.eta) + second.pt * np.sinh(second.eta)
    e1 = np.sqrt((first.pt * np.cosh(first.eta)) ** 2 + first.mass ** 2)
    e2 = np.sqrt((second.pt * np.cosh(second.eta)) ** 2 + second.mass ** 2)
    mass2 = (e1 + e2) ** 2 - px ** 2 - py ** 2 - pz ** 2
    return np.sqrt(ak.where(mass2 > 0, mass2, 0))

m_ee = pair_mass(e_minus, e_plus)
z_window = (m_ee > z_mass_min) & (m_ee < z_mass_max)

e_minus = ak.flatten(e_minus[z_window])
e_plus = ak.flatten(e_plus[z_window])
masses_ee = ak.to_numpy(ak.flatten(m_ee[z_window]))

print('Opposite-sign dielectron pairs in 70-110 GeV:', len(masses_ee))
print(f'Mean mass: {np.mean(masses_ee):.6f} GeV')
print(f'Median mass: {np.median(masses_ee):.6f} GeV')

```

```

Total selected electrons: 9734445
Events with >=2 selected electrons: 1903393
Opposite-sign dielectron pairs in 70-110 GeV: 1344681
Mean mass: 90.638089 GeV
Median mass: 91.148434 GeV

```

6 Build Detector-Level x-Space Array

```

[22]: def p4_array(particle):
    pt = ak.to_numpy(particle.pt)
    eta = ak.to_numpy(particle.eta)
    phi = ak.to_numpy(particle.phi)
    mass = ak.to_numpy(particle.mass)

    px = pt * np.cos(phi)
    py = pt * np.sin(phi)
    pz = pt * np.sinh(eta)
    energy = np.sqrt(px ** 2 + py ** 2 + pz ** 2 + mass ** 2)
    return np.stack([px, py, pz, energy], axis=1)

# Match ppzee ordering: [p_e-^mu, p_e+^mu], with p^mu = [px, py, pz, E]
x_data = np.concatenate([p4_array(e_minus), p4_array(e_plus)], axis=1)

```

```
print('x_data shape:', x_data.shape)
print('First x row:', x_data[0])
```

```
x_data shape: (1344681, 8)
First x row: [-35.98058   48.777832  35.590557  70.28917   20.748224 -27.40425
 26.977688  43.69528 ]
(1344681, 8)
First x row: [-35.98058   48.777832  35.590557  70.28917   20.748224 -27.40425
 26.977688  43.69528 ]
```

7 Load Theory-Level z-Space Prior

```
[23]: import h5py

# Load the MadGraph5 pre-detector physics prior z from the HDF5 file.
# Expected HDF5 layout:
#   FDL/zData : shape (N, 8)
# with columns:
#   [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]

if not theory_prior_file.exists():
    available_hdf5 = sorted([p.name for p in Path('.').glob('*.hdf5')])
    print('Available .hdf5 files in current working directory:', available_hdf5)
    raise FileNotFoundError(
        f'Cannot find {theory_prior_file}. '
        'Either move the HDF5 file into the notebook folder, '
        'or set theory_prior_file to the absolute Windows path.'
    )

with h5py.File(theory_prior_file, 'r') as f:
    print('HDF5 top-level keys:', list(f.keys()))
    if 'FDL' in f and isinstance(f['FDL'], h5py.Group) and 'zData' in f['FDL']:
        z_data = np.asarray(f['FDL/zData'])
    elif 'zData' in f:
        z_data = np.asarray(f['zData'])
    else:
        raise KeyError('Could not find z prior. Expected FDL/zData or zData in_
↳the HDF5 file.')

# Keep the same 8D ordering used by x_data:
# [p_e~mu, p_e+~mu], with p~mu = [px, py, pz, E]
z_data = z_data[:, :8]

assert x_data.ndim == 2 and x_data.shape[1] == 8, f'x_data should have shape_
↳(N, 8), got {x_data.shape}'
assert z_data.ndim == 2 and z_data.shape[1] == 8, f'z_data should have shape_
↳(N, 8), got {z_data.shape}'
```

```

assert np.all(np.isfinite(x_data)), 'x_data contains NaN or inf'
assert np.all(np.isfinite(z_data)), 'z_data contains NaN or inf'

# OTUS training uses unpaired samples from p(x) and p(z). Use the common sample
↪count.
num_samples = min(len(x_data), len(z_data))
x_data = x_data[:num_samples]
z_data = z_data[:num_samples]

x_dim = int(x_data.shape[1])
z_dim = int(z_data.shape[1])

print('Loaded z prior from:', theory_prior_file)
print('z_data shape:', z_data.shape)
print('x_data shape:', x_data.shape)
print('num_samples used:', num_samples)
print('First z row:', z_data[0])

```

```

HDF5 top-level keys: ['FDL']
Loaded z prior from: cms_dyee_mg5_8tev_fiducial_70_110.hdf5
z_data shape: (640449, 8)
x_data shape: (640449, 8)
num_samples used: 640449
First z row: [-37.5503      12.661246   24.690422   46.689926   37.5503    -12.661246
  98.864876  106.51102 ]

```

7.1 Pre-training sanity check

Compare the CMS detector-level mass distribution with the MG5 pre-detector prior before training. The MG5 curve should usually be narrower because it does not include detector resolution, electron energy smearing, or reconstruction effects.

```

[24]: from func_utils import Zboson_mass

x_m_before = Zboson_mass(x_data)
z_m_before = Zboson_mass(z_data)

fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(70, 110, 81)

ax.hist(x_m_before, bins=bins, histtype='step', linewidth=2, density=True,
↪label='CMS x')
ax.hist(z_m_before, bins=bins, histtype='step', linewidth=2, density=True,
↪label='MG5 z prior')
ax.axvline(91.1880, linestyle='--', linewidth=1.5, label=r'$m_Z = 91.1880$ GeV')

ax.set_xlabel(r'$m_{ee}$ [GeV]')
ax.set_ylabel('Normalized events')

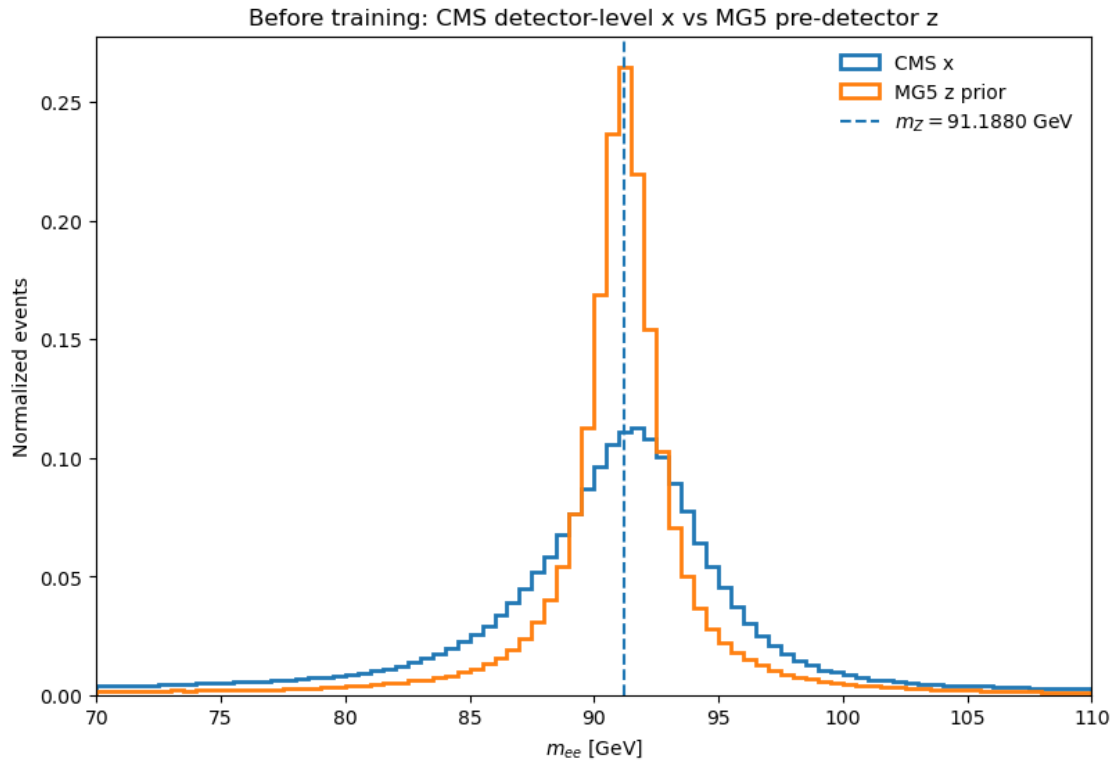
```

```

ax.set_xlim(70, 110)
ax.set_title('Before training: CMS detector-level x vs MG5 pre-detector z')
ax.legend(frameon=False)
plt.show()

print(f'CMS x mass mean: {np.mean(x_m_before):.4f} GeV')
print(f'MG5 z mass mean: {np.mean(z_m_before):.4f} GeV')
print(f'CMS x mass std: {np.std(x_m_before):.4f} GeV')
print(f'MG5 z mass std: {np.std(z_m_before):.4f} GeV')

```



```

CMS x mass mean: 90.6363 GeV
MG5 z mass mean: 91.0066 GeV
CMS x mass std: 5.8516 GeV
MG5 z mass std: 4.0704 GeV

```

7.1.1 Define target invariant masses

```

[25]: # Electrons are treated as massless for consistency with ppzee.ipynb.
x_inv_masses = np.zeros(2)
z_inv_masses = np.zeros(2)

```

8 Train/Test Split

```
[26]: from torch.utils.data import DataLoader

rng = np.random.default_rng(seed)
x_perm = rng.permutation(len(x_data))
z_perm = rng.permutation(len(z_data))
x_data = x_data[x_perm]
z_data = z_data[z_perm]

train_ratio = 0.80
val_ratio = 0.10

train_size = int(num_samples * train_ratio)
val_size = int(num_samples * val_ratio)
test_size = num_samples - train_size - val_size

x_train = x_data[:train_size]
x_val = x_data[train_size:train_size + val_size]
x_test = x_data[train_size + val_size:]

z_train = z_data[:train_size]
z_val = z_data[train_size:train_size + val_size]
z_test = z_data[train_size + val_size:]

x_train, x_val, x_test, z_train, z_val, z_test = list(map(
    lambda arr: arr.astype(float_type),
    [x_train, x_val, x_test, z_train, z_val, z_test],
))

x_train_mean, x_train_std = np.mean(x_train, axis=0), np.std(x_train, axis=0)
z_train_mean, z_train_std = np.mean(z_train, axis=0), np.std(z_train, axis=0)

eval_batch_size = min(20000, max(1, len(x_val)))
eval_loaders = DataLoader(dataset=x_val, batch_size=eval_batch_size,
    ↪shuffle=True), \
    DataLoader(dataset=z_val, batch_size=eval_batch_size,
    ↪shuffle=True)

print('z_train shape, x_train shape:', z_train.shape, x_train.shape)
print('z_val shape, x_val shape:', z_val.shape, x_val.shape)
print('z_test shape, x_test shape:', z_test.shape, x_test.shape)

z_train shape, x_train shape: (512359, 8) (512359, 8)
z_val shape, x_val shape: (64044, 8) (64044, 8)
z_test shape, x_test shape: (64046, 8) (64046, 8)
```

9 Train

9.1 Import Training Specific Libraries and Functions

```
[27]: from torch import optim
      from ppzee_utils import train_and_val
```

9.2 Central hyperparameter block

Edit this one cell when tuning architecture, batch sizes, and staged training.

```
[28]: # =====
# Central Hyperparameter Block
# Edit this cell when tuning the training.
# =====

HP = {
    # -----
    # Model architecture
    # -----
    "cond_noise": True,
    "num_hidden_layers": 2,
    "dim_per_hidden_layer": 256,
    "activation": torch.nn.ReLU,
    "sigma_fun": "softplus",

    "train_batch_size": 4096,
    "eval_batch_size": 20000,
    "preload_data_to_gpu": True,
    "pin_memory_if_cpu_loader": True,

    # -----
    # Training stages
    # beta: x reconstruction loss for  $x \rightarrow z \rightarrow x$ 
    # lamb: latent-prior matching loss between  $\text{encode}(x)$  and  $z$ 
    # tau: direct generator loss between  $\text{decode}(z)$  and CMS  $x$ 
    # nu_e / nu_d: anchor losses for encoder/decoder
    # -----
    "stages": [
        {
            "name": "stage1_anchor_warmup",
            "enabled": True,
            "epochs": 50,
            "lr": 1e-3,
            "beta": 1.0,
            "lamb": 1.0,
            "tau": 1.0,
            "rho": 0.0,
```

```

        "nu_e": 50.0,
        "nu_d": 50.0,
        "num_slices": 1000,
        "log_freq": 10,
        "lr_decay": False,
    },
    {
        "name": "stage2_joint_no_anchor",
        "enabled": True,
        "epochs": 100,
        "lr": 1e-3,
        "beta": 1.0,
        "lamb": 1.0,
        "tau": 1.0,
        "rho": 0.0,
        "nu_e": 0.0,
        "nu_d": 0.0,
        "num_slices": 1000,
        "log_freq": 20,
        "lr_decay": True,
    },
    {
        "name": "stage3_direct_z_to_x",
        "enabled": True,
        "epochs": 200,
        "lr": 1e-4,
        "beta": 0.5,
        "lamb": 0.5,
        "tau": 3.0,
        "rho": 0.0,
        "nu_e": 0.0,
        "nu_d": 0.0,
        "num_slices": 500,
        "log_freq": 20,
        "lr_decay": True,
    },
],
}

if HP["cond_noise"]:
    from models import CondNoiseAutoencoder
    Autoencoder = CondNoiseAutoencoder
else:
    from models import Autoencoder

print('Using model class:', Autoencoder.__name__)

```

```

print('Enabled stages:', [s['name'] for s in HP['stages'] if s.get('enabled',
↪True)])
print('train_batch_size:', HP['train_batch_size'])
print('preload_data_to_gpu:', HP['preload_data_to_gpu'])

```

Using model class: CondNoiseAutoencoder
 Enabled stages: ['stage1_anchor_warmup', 'stage2_joint_no_anchor',
 'stage3_direct_z_to_x']
 train_batch_size: 4096
 preload_data_to_gpu: True

9.3 Build model from hyperparameter block

```

[29]: # =====
# Build Model from HP
# =====

num_hidden_layers = HP["num_hidden_layers"]
dim_per_hidden_layer = HP["dim_per_hidden_layer"]
hidden_layer_dims = num_hidden_layers * [dim_per_hidden_layer]

activation = HP["activation"]
sigma_fun = HP["sigma_fun"]

model = Autoencoder(
    x_dim=x_dim,
    z_dim=z_dim,
    hidden_layer_dims=hidden_layer_dims,
    raw_io=True,
    x_stats=np.stack([x_train_mean, x_train_std]),
    z_stats=np.stack([z_train_mean, z_train_std]),
    x_inv_masses=x_inv_masses,
    z_inv_masses=z_inv_masses,
    stoch_enc=True,
    stoch_dec=True,
    activation=activation,
    sigma_fun=sigma_fun,
)

model

```

```

[29]: CondNoiseAutoencoder(
  (encoder): CondNoiseMLP(
    (sigma_fun): Softplus(beta=1.0, threshold=20.0)
    (output_nn): Sequential(
      (0): Linear(in_features=14, out_features=256, bias=True)
      (1): ReLU()
    )
  )

```

```

        (2): Linear(in_features=256, out_features=256, bias=True)
        (3): ReLU()
        (4): Linear(in_features=256, out_features=6, bias=True)
    )
    (cond_noise_nn): Sequential(
      (0): Linear(in_features=8, out_features=256, bias=True)
      (1): ReLU()
      (2): Linear(in_features=256, out_features=256, bias=True)
      (3): ReLU()
      (4): Linear(in_features=256, out_features=12, bias=True)
    )
  )
  (decoder): CondNoiseMLP(
    (sigma_fun): Softplus(beta=1.0, threshold=20.0)
    (output_nn): Sequential(
      (0): Linear(in_features=14, out_features=256, bias=True)
      (1): ReLU()
      (2): Linear(in_features=256, out_features=256, bias=True)
      (3): ReLU()
      (4): Linear(in_features=256, out_features=6, bias=True)
    )
    (cond_noise_nn): Sequential(
      (0): Linear(in_features=8, out_features=256, bias=True)
      (1): ReLU()
      (2): Linear(in_features=256, out_features=256, bias=True)
      (3): ReLU()
      (4): Linear(in_features=256, out_features=12, bias=True)
    )
  )
)
)

```

```

[30]: from func_utils import sliced_wd

P = 2
lambda_mass = 0.7

def dilepton_mass_torch(z, eps=1e-6):
    # z columns:
    # [e- px, e- py, e- pz, e- E, e+ px, e+ py, e+ pz, e+ E]
    px = z[:, 0] + z[:, 4]
    py = z[:, 1] + z[:, 5]
    pz = z[:, 2] + z[:, 6]
    E = z[:, 3] + z[:, 7]

    m2 = E**2 - px**2 - py**2 - pz**2
    return torch.sqrt(torch.clamp(m2, min=eps))

```

```

def wasserstein_1d_sorted(a, b, p=2):
    a = torch.sort(a.reshape(-1))[0]
    b = torch.sort(b.reshape(-1))[0]

    n = min(a.numel(), b.numel())
    a = a[:n]
    b = b[:n]

    return torch.mean(torch.abs(a - b)**p)

# Use MG5 prior mass scale for normalization.
# This keeps the mass loss numerically stable.
with torch.no_grad():
    z_train_torch = torch.tensor(z_train, dtype=torch.float32)
    m_z_train = dilepton_mass_torch(z_train_torch)
    m_z_mean = m_z_train.mean().item()
    m_z_std = m_z_train.std().item()

print("MG5 z mass mean/std:", m_z_mean, m_z_std)

def z_loss_with_mass(z, z_tilde):
    # Original 8D z-space sliced Wasserstein loss
    loss_z_8d = sliced_wd(z, z_tilde, config['num_slices'], P)

    # Extra 1D mass matching loss
    m_z = dilepton_mass_torch(z)
    m_z_tilde = dilepton_mass_torch(z_tilde)

    # Normalize by MG5 mass scale
    m_z = (m_z - m_z_mean) / m_z_std
    m_z_tilde = (m_z_tilde - m_z_mean) / m_z_std

    loss_mass = wasserstein_1d_sorted(m_z, m_z_tilde, p=P)

    return loss_z_8d + lambda_mass * loss_mass

```

MG5 z mass mean/std: 91.00443267822266 4.065318584442139

9.4 Train OTUS

```

[31]: # =====
# Fast DataLoaders from HP
# =====
# The old version used PyTorch DataLoader directly on NumPy arrays.
# That works, but it creates/collates CPU tensors batch-by-batch.

```

```

# For this tabular 8D dataset, it is faster to make torch tensors once.
# If CUDA is available, we keep train/val tensors on the GPU once.
# Precision is unchanged because torch.as_tensor keeps the array dtype.

class FastTensorLoader:
    """Minimal batch loader for one tensor.

    It is compatible with the OTUS train_and_val code as long as the code only
    needs iteration and len(loader). It returns one tensor batch at a time.
    """
    def __init__(self, tensor, batch_size, shuffle=True, drop_last=False):
        self.tensor = tensor
        self.batch_size = int(batch_size)
        self.shuffle = bool(shuffle)
        self.drop_last = bool(drop_last)
        self.n = int(tensor.shape[0])
        self.dataset = tensor # for compatibility with code that checks loader.
        ↪ dataset

    def __len__(self):
        if self.drop_last:
            return self.n // self.batch_size
        return (self.n + self.batch_size - 1) // self.batch_size

    def __iter__(self):
        if self.shuffle:
            idx = torch.randperm(self.n, device=self.tensor.device)
        else:
            idx = torch.arange(self.n, device=self.tensor.device)

        stop = self.n if not self.drop_last else (self.n // self.batch_size) *
        ↪ self.batch_size
        for start in range(0, stop, self.batch_size):
            batch_idx = idx[start:start + self.batch_size]
            yield self.tensor.index_select(0, batch_idx)

def make_training_tensor(arr, name, loader_device, pin_cpu_memory=False):
    t = torch.as_tensor(arr) # preserve dtype from float_type; no precision
    ↪ change
    if loader_device.type == 'cuda':
        t = t.to(loader_device)
    elif pin_cpu_memory and torch.cuda.is_available():
        t = t.pin_memory()
    print(f'{name}: shape={tuple(t.shape)}, dtype={t.dtype}, device={t.device}')
    return t

```

```

train_batch_size = min(HP["train_batch_size"], max(1, len(x_train)))
eval_batch_size = min(HP["eval_batch_size"], max(1, len(x_val)))

device_obj = torch.device(device)
preload_to_gpu = bool(HP.get("preload_data_to_gpu", True)) and (device_obj.type
    == 'cuda')
loader_device = device_obj if preload_to_gpu else torch.device('cpu')
pin_cpu_memory = bool(HP.get("pin_memory_if_cpu_loader", True)) and
    (loader_device.type == 'cpu') and torch.cuda.is_available()

print('loader_device:', loader_device)
print('pin_cpu_memory:', pin_cpu_memory)

x_train_loader = make_training_tensor(x_train, 'x_train_loader', loader_device,
    pin_cpu_memory)
z_train_loader = make_training_tensor(z_train, 'z_train_loader', loader_device,
    pin_cpu_memory)
x_val_loader = make_training_tensor(x_val, 'x_val_loader', loader_device,
    pin_cpu_memory)
z_val_loader = make_training_tensor(z_val, 'z_val_loader', loader_device,
    pin_cpu_memory)

train_loaders = (
    FastTensorLoader(x_train_loader, batch_size=train_batch_size, shuffle=True),
    FastTensorLoader(z_train_loader, batch_size=train_batch_size, shuffle=True),
)

eval_loaders = (
    FastTensorLoader(x_val_loader, batch_size=eval_batch_size, shuffle=False),
    FastTensorLoader(z_val_loader, batch_size=eval_batch_size, shuffle=False),
)

print('train_batch_size:', train_batch_size)
print('eval_batch_size:', eval_batch_size)
print('training batches per epoch:', min(len(train_loaders[0]),
    len(train_loaders[1])))
print('eval batches:', min(len(eval_loaders[0]), len(eval_loaders[1])))

```

```

loader_device: cuda
pin_cpu_memory: False
x_train_loader: shape=(512359, 8), dtype=torch.float32, device=cuda:0
z_train_loader: shape=(512359, 8), dtype=torch.float32, device=cuda:0
x_val_loader: shape=(64044, 8), dtype=torch.float32, device=cuda:0
z_val_loader: shape=(64044, 8), dtype=torch.float32, device=cuda:0
train_batch_size: 4096
eval_batch_size: 20000
training batches per epoch: 126

```

eval_batches: 4

```
[32]: %%time

# =====
# Train OTUS using stages from HP
# =====

history = None
eval_losses = None

for stage in HP["stages"]:
    if not stage.get("enabled", True):
        print(f"Skipping disabled stage: {stage['name']}")
        continue

    print("\n" + "=" * 80)
    print("Starting training stage:", stage["name"])
    print("=" * 80)

    config = {
        "num_hidden_layers": HP["num_hidden_layers"],
        "dim_per_hidden_layer": HP["dim_per_hidden_layer"],
        "epochs": stage["epochs"],
        "lr": stage["lr"],
        "beta": stage["beta"],
        "lamb": stage["lamb"],
        "tau": stage["tau"],
        "rho": stage["rho"],
        "nu_e": stage["nu_e"],
        "nu_d": stage["nu_d"],
        "num_slices": stage["num_slices"],
    }

    optimizer = optim.Adam(model.parameters(), lr=config["lr"])

    eval_losses, history = train_and_val(
        model,
        train_loaders,
        eval_loaders,
        config,
        optimizer,
        verbose=True,
        prev_hist=history,
        log_freq=stage["log_freq"],
        lr_decay=stage["lr_decay"],
        z_loss_fun=z_loss_with_mass,
```

```
)

print("\nTraining complete.")
print("Final eval losses:", eval_losses)
```

```
=====
Starting training stage: stage1_anchor_warmup
=====
```

```
{'num_hidden_layers': 2, 'dim_per_hidden_layer': 256, 'epochs': 50, 'lr': 0.001,
'beta': 1.0, 'lamb': 1.0, 'tau': 1.0, 'rho': 0.0, 'nu_e': 50.0, 'nu_d': 50.0,
'num_slices': 1000}
epoch: 0
train -- loss:365.68, x_loss:76.8729, z_loss:75.9512, alt_x_loss:71.1725,
x_constraint_loss:0, anchor_loss:2.83366
eval -- loss:87.3426, x_loss:103.736, z_loss:38.8924, alt_x_loss:48.4503,
anchor_loss:2.86577
epoch: 10
train -- loss:290.725, x_loss:8.03234, z_loss:91.7688, alt_x_loss:131.283,
x_constraint_loss:0, anchor_loss:1.19281
eval -- loss:58.4266, x_loss:20.5053, z_loss:26.7062, alt_x_loss:31.7204,
anchor_loss:1.30256
epoch: 20
train -- loss:253.674, x_loss:13.1006, z_loss:113.281, alt_x_loss:90.5005,
x_constraint_loss:0, anchor_loss:0.735837
eval -- loss:56.3303, x_loss:37.9631, z_loss:8.93405, alt_x_loss:47.3962,
anchor_loss:0.775625
epoch: 30
train -- loss:212.809, x_loss:13.2155, z_loss:84.0424, alt_x_loss:93.6739,
x_constraint_loss:0, anchor_loss:0.437541
eval -- loss:34.8092, x_loss:23.2482, z_loss:6.82169, alt_x_loss:27.9875,
anchor_loss:0.512401
epoch: 40
train -- loss:151.937, x_loss:10.0812, z_loss:38.1514, alt_x_loss:90.4994,
x_constraint_loss:0, anchor_loss:0.264098
eval -- loss:33.8072, x_loss:10.0265, z_loss:11.5986, alt_x_loss:22.2086,
anchor_loss:0.303443
epoch: 49
train -- loss:220.814, x_loss:8.08036, z_loss:84.7473, alt_x_loss:120.586,
x_constraint_loss:0, anchor_loss:0.147997
eval -- loss:40.0503, x_loss:14.5591, z_loss:18.0423, alt_x_loss:22.008,
anchor_loss:0.149287
```

```
=====
Starting training stage: stage2_joint_no_anchor
=====
```

```
{'num_hidden_layers': 2, 'dim_per_hidden_layer': 256, 'epochs': 100, 'lr':
0.001, 'beta': 1.0, 'lamb': 1.0, 'tau': 1.0, 'rho': 0.0, 'nu_e': 0.0, 'nu_d':
```

```

0.0, 'num_slices': 1000}
epoch: 0
train -- loss:332.341, x_loss:7.00881, z_loss:70.8897, alt_x_loss:254.442,
x_constraint_loss:0, anchor_loss:0
eval -- loss:28.4225, x_loss:7.60404, z_loss:5.33366, alt_x_loss:23.0888,
anchor_loss:0.198888
epoch: 20
train -- loss:203.208, x_loss:4.4996, z_loss:100.334, alt_x_loss:98.3746,
x_constraint_loss:0, anchor_loss:0
eval -- loss:26.6819, x_loss:4.26639, z_loss:7.82479, alt_x_loss:18.8571,
anchor_loss:0.328287
epoch: 40
train -- loss:152.037, x_loss:2.82231, z_loss:37.2045, alt_x_loss:112.01,
x_constraint_loss:0, anchor_loss:0
eval -- loss:30.2569, x_loss:3.2631, z_loss:9.21836, alt_x_loss:21.0385,
anchor_loss:0.348202
epoch: 60
train -- loss:194.792, x_loss:2.53221, z_loss:57.1895, alt_x_loss:135.07,
x_constraint_loss:0, anchor_loss:0
eval -- loss:21.0445, x_loss:3.27703, z_loss:2.34189, alt_x_loss:18.7026,
anchor_loss:0.354742
epoch: 80
train -- loss:133.071, x_loss:2.44614, z_loss:38.3504, alt_x_loss:92.2746,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.2141, x_loss:3.14277, z_loss:3.42703, alt_x_loss:16.7871,
anchor_loss:0.36361
epoch: 99
train -- loss:118.572, x_loss:1.64812, z_loss:58.521, alt_x_loss:58.4025,
x_constraint_loss:0, anchor_loss:0
eval -- loss:22.7471, x_loss:2.14461, z_loss:4.81239, alt_x_loss:17.9348,
anchor_loss:0.361378

```

```

=====
Starting training stage: stage3_direct_z_to_x
=====

```

```

{'num_hidden_layers': 2, 'dim_per_hidden_layer': 256, 'epochs': 200, 'lr':
0.0001, 'beta': 0.5, 'lamb': 0.5, 'tau': 3.0, 'rho': 0.0, 'nu_e': 0.0, 'nu_d':
0.0, 'num_slices': 500}
epoch: 0
train -- loss:312.832, x_loss:2.33883, z_loss:54.7753, alt_x_loss:94.7584,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.2889, x_loss:2.77987, z_loss:2.14069, alt_x_loss:18.1482,
anchor_loss:0.364057
epoch: 20
train -- loss:343.276, x_loss:2.13957, z_loss:128.161, alt_x_loss:92.7085,
x_constraint_loss:0, anchor_loss:0
eval -- loss:20.0049, x_loss:2.17005, z_loss:2.70082, alt_x_loss:17.3041,
anchor_loss:0.364571

```

```

epoch: 40
train -- loss:282.666, x_loss:1.63706, z_loss:81.9179, alt_x_loss:80.2961,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.9775, x_loss:2.14323, z_loss:2.57624, alt_x_loss:17.4012,
anchor_loss:0.365138
epoch: 60
train -- loss:302.221, x_loss:1.97625, z_loss:31.2872, alt_x_loss:95.1965,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.3817, x_loss:2.09157, z_loss:2.48051, alt_x_loss:16.9012,
anchor_loss:0.364184
epoch: 80
train -- loss:330.037, x_loss:2.09777, z_loss:45.455, alt_x_loss:102.087,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.5707, x_loss:2.11838, z_loss:2.68783, alt_x_loss:16.8829,
anchor_loss:0.363138
epoch: 100
train -- loss:562.764, x_loss:1.51404, z_loss:67.132, alt_x_loss:176.147,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.6462, x_loss:2.08711, z_loss:2.27219, alt_x_loss:17.374,
anchor_loss:0.362902
epoch: 120
train -- loss:255.546, x_loss:2.0556, z_loss:42.8651, alt_x_loss:77.6953,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.7605, x_loss:2.11071, z_loss:2.48401, alt_x_loss:17.2765,
anchor_loss:0.362426
epoch: 140
train -- loss:273.033, x_loss:1.49833, z_loss:88.9332, alt_x_loss:75.9391,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.7695, x_loss:2.10172, z_loss:2.2965, alt_x_loss:17.473,
anchor_loss:0.361936
epoch: 160
train -- loss:390.852, x_loss:1.8156, z_loss:75.9158, alt_x_loss:117.329,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.524, x_loss:2.09638, z_loss:2.48961, alt_x_loss:17.0344,
anchor_loss:0.361008
epoch: 180
train -- loss:425.23, x_loss:1.70172, z_loss:90.1797, alt_x_loss:126.43,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.1392, x_loss:2.13006, z_loss:2.3006, alt_x_loss:16.8386,
anchor_loss:0.362053
epoch: 199
train -- loss:384.988, x_loss:1.69101, z_loss:40.8741, alt_x_loss:121.235,
x_constraint_loss:0, anchor_loss:0
eval -- loss:19.2913, x_loss:2.11181, z_loss:2.4454, alt_x_loss:16.8459,
anchor_loss:0.361489

```

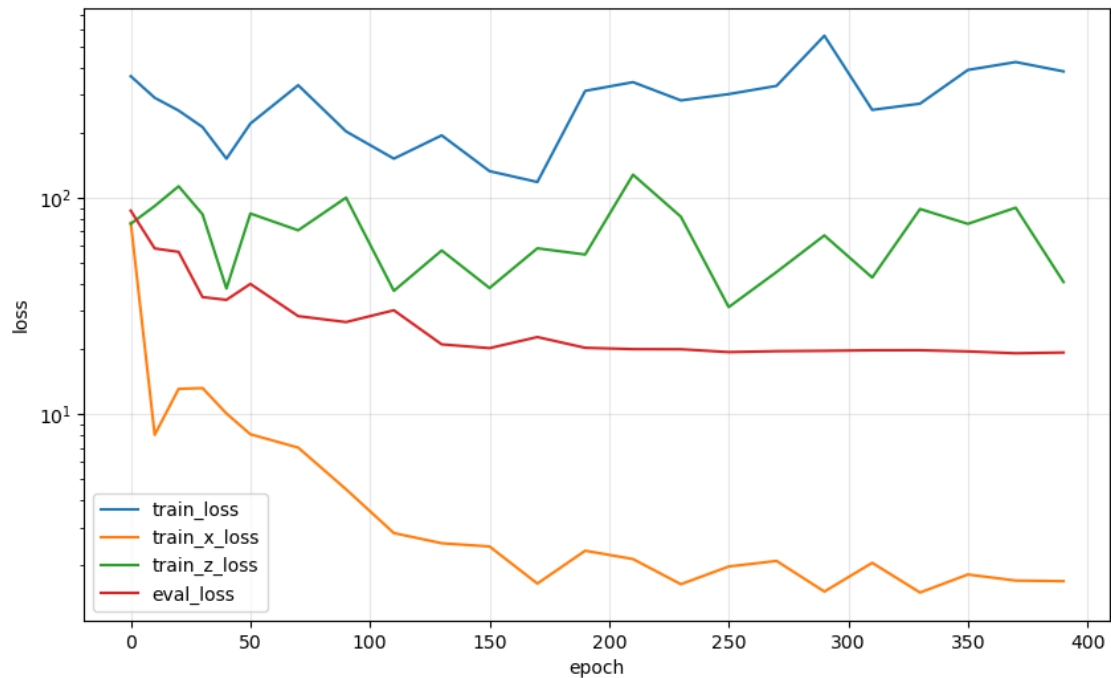
Training complete.

Final eval losses: {'x_loss': 2.1118085384368896, 'z_loss': 2.445399045944214,

```
'alt_x_loss': 16.845947265625, 'encoder_anchor_loss': 0.18099050223827362,  
'decoder_anchor_loss': 0.18049831688404083, 'loss': 19.291346311569214}  
CPU times: total: 9min 9s  
Wall time: 11min 4s
```

9.5 Plot Loss History

```
[33]: def history_to_numpy(values):  
    """Convert history lists containing Python numbers, NumPy values, or Torch_  
    ↪ensors to NumPy."""  
    out = []  
    for v in values:  
        if isinstance(v, torch.Tensor):  
            out.append(v.detach().cpu().item() if v.ndim == 0 else v.detach().  
            ↪cpu().numpy())  
        else:  
            out.append(v)  
    return np.asarray(out, dtype=float)  
  
epoch = history_to_numpy(history['epoch'])  
train_loss = history_to_numpy(history['train_loss'])  
train_x_loss = history_to_numpy(history['train_x_loss'])  
train_z_loss = history_to_numpy(history['train_z_loss'])  
eval_loss = history_to_numpy(history['eval_loss'])  
  
fig = plt.figure(figsize=(10, 6))  
  
plt.plot(epoch, train_loss, label='train_loss')  
plt.plot(epoch, train_x_loss, label='train_x_loss')  
plt.plot(epoch, train_z_loss, label='train_z_loss')  
plt.plot(epoch, eval_loss, label='eval_loss')  
  
plt.xlabel('epoch')  
plt.ylabel('loss')  
plt.yscale('log')  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.show()
```



10 Show Results

10.1 Get Results

```
[34]: ## Reset random seeds ##
torch.manual_seed(seed)
np.random.seed(seed)

## Convert model to CPU and evaluation mode ##
model.to('cpu')
model.encoder.output_stats.to('cpu')
model.decoder.output_stats.to('cpu')
model.eval()

all_arrs = {'train': {}, 'val': {}, 'test': {}}
all_arrs['train']['x'] = x_train
all_arrs['train']['z'] = z_train
all_arrs['val']['x'] = x_val
all_arrs['val']['z'] = z_val
all_arrs['test']['x'] = x_test
all_arrs['test']['z'] = z_test

with torch.no_grad():
    for data_key in 'train', 'val', 'test':
```

```

arrs = all_arrs[data_key]

z_tensor = torch.from_numpy(arrs['z'])
x_tensor = torch.from_numpy(arrs['x'])

arrs['z_decoded'] = model.decode(z_tensor)
arrs['x_encoded'] = model.encode(x_tensor)
arrs['x_reconstructed'] = model.decode(arrs['x_encoded'])

for field, arr in list(arrs.items()):
    if isinstance(arr, torch.Tensor):
        arrs[field] = arr.detach().cpu().numpy()

print('Generated arrays for:', list(all_arrs.keys()))
for key in ['x', 'z', 'x_reconstructed', 'z_decoded']:
    print(key, all_arrs['val'][key].shape)

```

```

Generated arrays for: ['train', 'val', 'test']
x (64044, 8)
z (64044, 8)
x_reconstructed (64044, 8)
z_decoded (64044, 8)

```

11 Save Results

```

[35]: from pathlib import Path

# Save everything under saves/<dataset_name>/
save_dir = Path("saves") / dataset_name
save_dir.mkdir(parents=True, exist_ok=True)

# 1. Save state_dict weights
model_save_path = save_dir / "model_state_dict.pkl"
torch.save(model.state_dict(), model_save_path)
print("Saved model weights to", model_save_path)

# 2. Save full model object, so diagnostic notebooks can directly call model.
    ↪ encode(x)
full_model_save_path = save_dir / "model_full.pt"
torch.save(model, full_model_save_path)
print("Saved full model to", full_model_save_path)

# 3. Save test results
results_save_path = save_dir / "results_test.npz"
np.savez(results_save_path, **all_arrs["test"])
print("Model results saved at", results_save_path)

```

```
# 4. Save training history
history_save_path = save_dir / "history.npz"
history_np = {key: history_to_numpy(value) for key, value in history.items()}
np.savez(history_save_path, **history_np)
print("Training history saved at", history_save_path)
```

Saved model weights to
saves\cms_doubleelectron_mg5_dyee_XL_lambda0.7\model_state_dict.pkl
Saved full model to saves\cms_doubleelectron_mg5_dyee_XL_lambda0.7\model_full.pt
Model results saved at
saves\cms_doubleelectron_mg5_dyee_XL_lambda0.7\results_test.npz
Training history saved at
saves\cms_doubleelectron_mg5_dyee_XL_lambda0.7\history.npz

12 Validation Plots

```
[36]: from func_utils import Zboson_mass

arrs = all_arrs['val']

m_x = Zboson_mass(arrs['x'])
m_rec = Zboson_mass(arrs['x_reconstructed'])
m_zdec = Zboson_mass(arrs['z_decoded'])

def summarize_mass(name, m):
    bins_summary = np.linspace(70, 110, 81)
    counts, edges = np.histogram(m, bins=bins_summary)
    peak = 0.5 * (edges[np.argmax(counts)] + edges[np.argmax(counts) + 1])
    print(
        f'{name:12s} '
        f'mean={np.mean(m):7.3f}, '
        f'std={np.std(m):7.3f}, '
        f'peak={peak:7.3f}, '
        f'frac_70_110={np.mean((m > 70) & (m < 110)):6.3f}'
    )

summarize_mass('CMS x', m_x)
summarize_mass('x-z-x', m_rec)
summarize_mass('z-x', m_zdec)

fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(70, 110, 81)

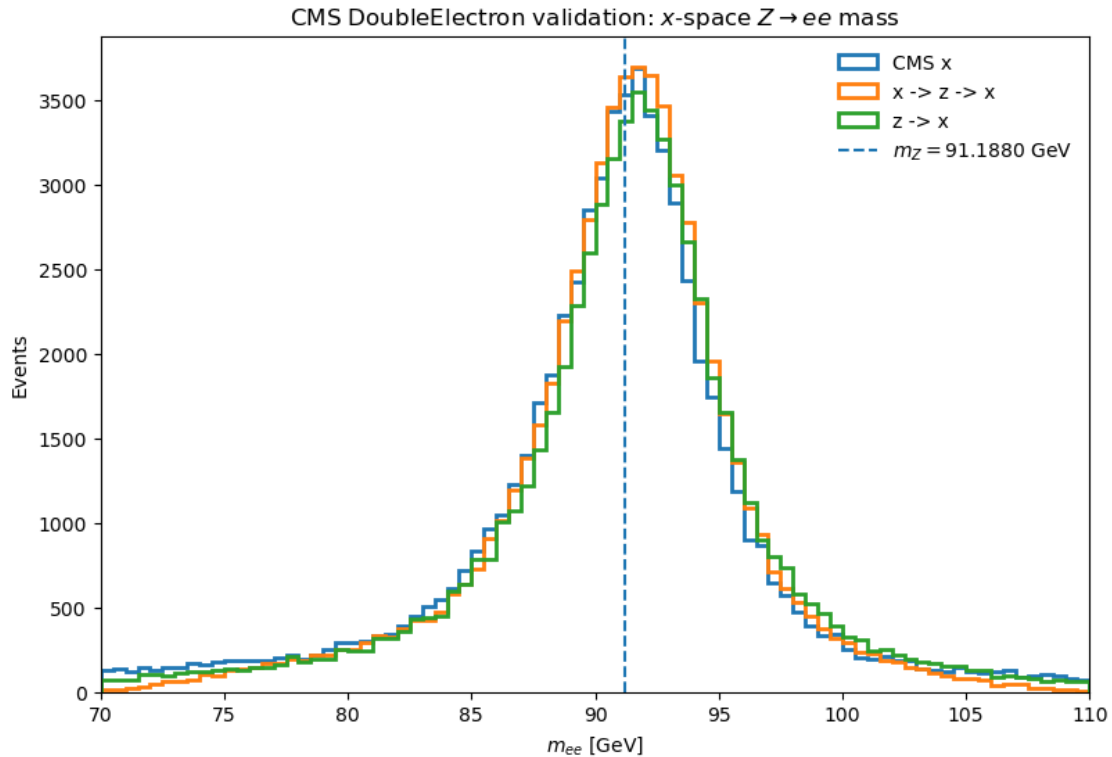
ax.hist(m_x, bins=bins, histtype='step', linewidth=2, label='CMS x')
ax.hist(m_rec, bins=bins, histtype='step', linewidth=2, label='x -> z -> x')
ax.hist(m_zdec, bins=bins, histtype='step', linewidth=2, label='z -> x')
ax.axvline(91.1880, linestyle='--', linewidth=1.5, label=r'$m_Z = 91.1880$ GeV')
```

```

ax.set_xlabel(r'$m_{ee}$ [GeV]')
ax.set_ylabel('Events')
ax.set_xlim(70, 110)
ax.set_title(r'CMS DoubleElectron validation: $x$-space $Z \to ee$ mass')
ax.legend(frameon=False)
plt.show()

```

CMS x	mean= 90.593, std= 5.866, peak= 91.750, frac_70_110= 1.000
x-z-x	mean= 90.955, std= 5.222, peak= 91.750, frac_70_110= 0.998
z-x	mean= 91.127, std= 6.731, peak= 91.750, frac_70_110= 0.981



12.1 Optional generator diagnostic: dilepton transverse momentum

A pure LO MG5 $pp \rightarrow e^+ e^-$ prior has very small dilepton recoil before OTUS. This check helps diagnose whether the generated $z \rightarrow x$ events learn CMS-like transverse recoil.

```

[37]: def dilepton_pt(y):
    px = y[:, 0] + y[:, 4]
    py = y[:, 1] + y[:, 5]
    return np.sqrt(px**2 + py**2)

x_ptll = dilepton_pt(arrs['x'])

```

```

zdec_ptll = dilepton_pt(arrs['z_decoded'])

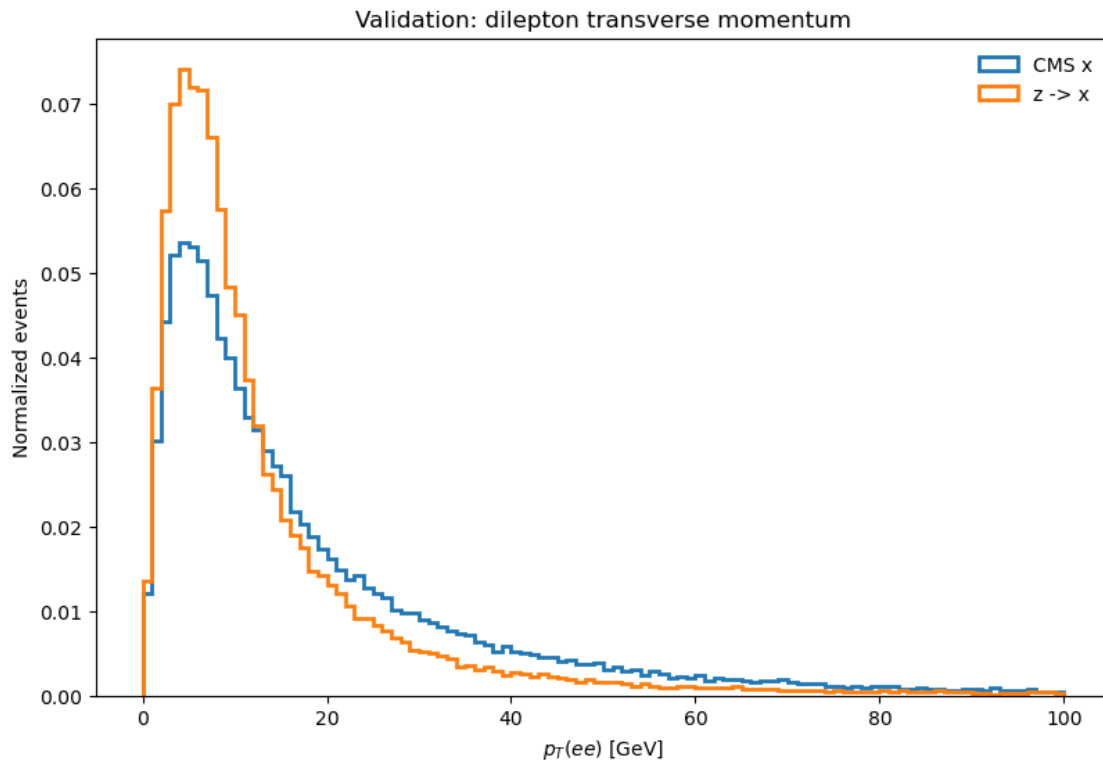
fig, ax = plt.subplots(figsize=(8, 5.5), constrained_layout=True)
bins = np.linspace(0, 100, 101)

ax.hist(x_ptll, bins=bins, histtype='step', linewidth=2, density=True,
        ↪label='CMS x')
ax.hist(zdec_ptll, bins=bins, histtype='step', linewidth=2, density=True,
        ↪label='z -> x')

ax.set_xlabel(r'$p_T(ee)$ [GeV]')
ax.set_ylabel('Normalized events')
ax.set_title(r'Validation: dilepton transverse momentum')
ax.legend(frameon=False)
plt.show()

print(f'CMS pT(ee) mean: {np.mean(x_ptll):.4f} GeV')
print(f'z -> x pT(ee) mean: {np.mean(zdec_ptll):.4f} GeV')

```

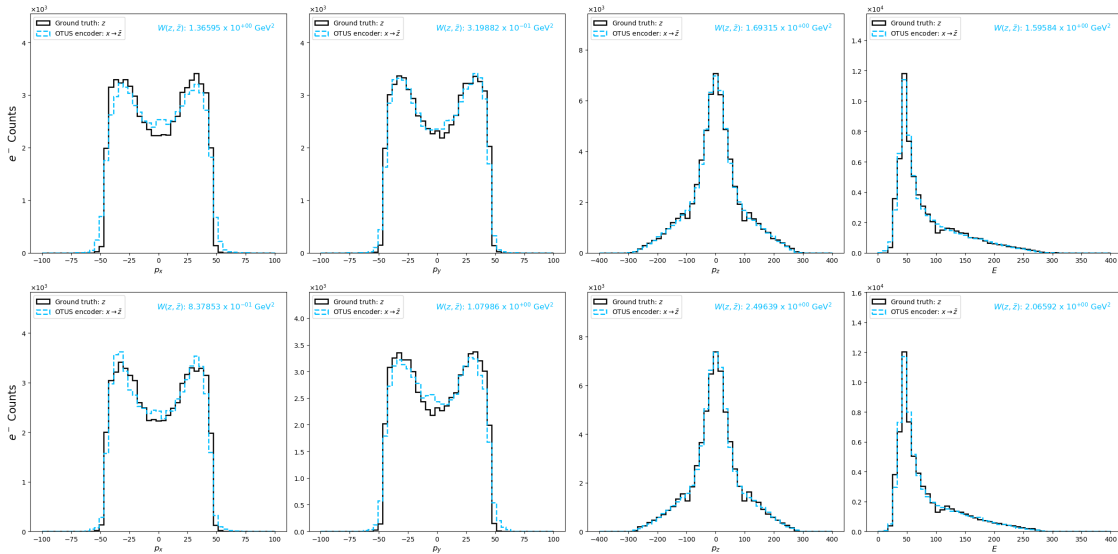


CMS pT(ee) mean: 20.7032 GeV
 z -> x pT(ee) mean: 14.3322 GeV

```
[38]: from plot_utils import plotFunction

dataList = [arrs['z'], arrs['x_encoded']]
pltDim = (2, 4)
numBins = 50
binsList = [np.linspace(-100., 100., numBins),
             np.linspace(-100., 100., numBins),
             np.linspace(-400., 400., numBins),
             np.linspace(0., 400., numBins),
             np.linspace(-100., 100., numBins),
             np.linspace(-100., 100., numBins),
             np.linspace(-400., 400., numBins),
             np.linspace(0., 400., numBins)]
particleNameList = [r'$e^-$'] * 4 + [r'$e^+$'] * 4
nameList = [r'$p_x$', r'$p_y$', r'$p_z$', r'$E$', r'$p_x$', r'$p_y$', r'$p_z$',
            r'$E$']

plotFunction(dataList=dataList, pltDim=pltDim, binsList=binsList,
             particleNameList=particleNameList, nameList=nameList)
```



```
[39]: dataList = [arrs['x'], arrs['x_reconstructed'], arrs['z_decoded']]

plotFunction(dataList=dataList, pltDim=pltDim, binsList=binsList,
             particleNameList=particleNameList, nameList=nameList)
```

